

LangChain & Agentic AI – From Basics to Advanced 18

=====

Good Schema Design Rules:

1. Keep fields clear. ie meaningful field names we required to use

```
class Student(BaseModel):
```

```
    x : str
```

```
    y: int
```

```
class Student(BaseModel):
```

```
    student_name : str
```

```
    age: int
```

2. Highly recommended to use descriptions

```
age: int = Field(
```

```
description="age of student"
```

```
)
```

3. Choose Correct Data Types

4. Make truly missing fields optional and provide default values

```
course: str | None = Field(  
    default="Python",  
    description="Name of the course"  
)
```

5. Dont create unnecessary fields

33. PROJECT 1 — STUDENT INFORMATION EXTRACTOR

Requirement:

User gives:

Ravi is 22 years old.

He is learning Python.

His email is ravi@example.com.

Our program needs:

name

age

course

email

34. Student Extractor Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Student(BaseModel):
```

```
    name: str = Field(
```

```
        description="Name of the student"
```

```
    )
```

```
age: int | None = Field(  
    default=None,  
    description="Age if available"  
)
```

```
course: str | None = Field(  
    default=None,  
    description="Course the student is learning"  
)
```

```
email: str | None = Field(  
    default=None,  
    description="Email address if available"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(Student)
```

```
text = input(  
    "Enter student information: "  
)
```

```
prompt = f"""
```

```
Extract student information.
```

```
Rules:
```

- Use only the supplied information.
- Do not invent missing values.
- Use null for optional information that is unavailable.

```
Student Information:
```

```
{text}
```

```
"""
```

```
student = structured_llm.invoke(prompt)
```

```
print("\nSTUDENT INFORMATION")
```

```
print("-----")
```

```
print("Name  :", student.name)
```

```
print("Age   :", student.age)
```

```
print("Course :", student.course)
```

```
print("Email  :", student.email)
```

```
=====
```

36. PROJECT 2 — RESUME INFORMATION EXTRACTOR

```
-----
```

Suppose the resume text says:

Suresh is a Java developer.

He has 7 years of experience.

He knows Java, Spring Boot, SQL and AWS.

We want:

name

job_title

experience

skills

email

38. Complete Resume Extractor

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class ResumeInfo(BaseModel):
```

```
    name: str | None = Field(
        default=None,
        description="Candidate name"
    )
```

```
    job_title: str | None = Field(
        default=None,
        description="Candidate job title if available"
```

)

```
experience: int | None = Field(  
    default=None,  
    description="Total experience in years if available"  
)
```

```
skills: list[str] = Field(  
    description="Skills explicitly mentioned in resume"  
)
```

```
email: str | None = Field(  
    default=None,  
    description="Candidate email if available"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(
    ResumeInfo
)
```

```
resume = input(
    "Enter resume information: "
)
```

```
prompt = f"""
```

```
Extract information from the resume text.
```

```
Important:
```

- Use only information present in the text.
- Do not guess missing information.
- Return missing optional values as null.

```
Resume:
```

```
{resume}
```

```
"""
```

```
result = structured_llm.invoke(prompt)
```

```
print("\nRESUME INFORMATION")
```

```
print("-----")
```

```
print("Name      :", result.name)
```

```
print("Job Title :", result.job_title)
```

```
print("Experience :", result.experience)
```

```
print("Skills    :", result.skills)
```

```
print("Email     :", result.email)
```

40. PROJECT 3 — PRODUCT INFORMATION EXTRACTOR

Suppose input:

The ABC laptop costs ₹65,000.

It has 16 GB RAM and 512 GB SSD.

We want:

name

price

ram

storage

availability

42. Product Extractor Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class Product(BaseModel):
```

```
    name: str = Field()
```

```
    description="Name of the product"
```

)

```
price: float | None = Field(  
    default=None,  
    description="Product price if mentioned"  
)
```

```
ram: str | None = Field(  
    default=None,  
    description="RAM specification if mentioned"  
)
```

```
storage: str | None = Field(  
    default=None,  
    description="Storage specification if mentioned"  
)
```

```
availability: bool | None = Field(  
    default=None,
```

```
        description="Whether the product is available, only if  
explicitly stated"
```

```
)
```

```
llm = ChatOpenAI(
```

```
    model="gpt-5.6-luna",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
structured_llm = llm.with_structured_output(Product)
```

```
text = input(
```

```
    "Enter product information: "
```

```
)
```

```
prompt = f"""
```

```
Extract product information.
```

```
Use only information explicitly available.
```

```
Do not guess missing information.
```

Product Text:

{text}

"""

```
product = structured_llm.invoke(prompt)
```

```
print(product)
```

```
print("\nAs Dictionary:")
```

```
print(product.model_dump())
```

```
print("\nAs JSON:")
```

```
print(product.model_dump_json())
```

43. PROJECT 4 — SENTIMENT ANALYZER

Suppose customer writes:

The course was excellent.

The explanations were easy to understand.

Our application needs:

sentiment

confidence

reason

44. Complete Sentiment Analyzer

```
import os
```

```
from typing import Literal
```

```
from langchain_openai import ChatOpenAI
```

```
from pydantic import BaseModel, Field
```

```
class SentimentResult(BaseModel):
```

```
    sentiment: Literal[
```

```
        "positive",
```

```
        "negative",
```

```
    "neutral"  
] = Field(  
    description="Overall sentiment of the text"  
)
```

```
confidence: float = Field(  
    ge=0,  
    le=1,  
    description="Confidence score between 0 and 1"  
)
```

```
reason: str = Field(  
    description="Very short reason for the sentiment"  
)
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
structured_llm = llm.with_structured_output(
    SentimentResult
)
```

```
text = input(
    "Enter customer feedback: "
)
```

```
result = structured_llm.invoke(text)
```

```
print("\nSENTIMENT ANALYSIS")
print("-----")
```

```
print("Sentiment :", result.sentiment)
print("Confidence :", result.confidence)
print("Reason    :", result.reason)
```

```
=====
```

MCQ 1

When include_raw=True, what is returned?

A. Only string

- B. Only AIMessage
- C. Dictionary containing raw, parsed, and parsing_error
- D. PromptTemplate

Ans: C

MCQ 2

What normally appears in:

`result["parsed"]`

when we supplied a Pydantic schema and parsing succeeded?

- A. Prompt
- B. Parsed Pydantic object
- C. API key
- D. Input text only

Ans: B

MCQ 3

If:

`result["parsing_error"]` is `None`

what does it mean?

A. Parsing succeeded

B. API key missing

C. Model was deleted

D. Schema is a string

Ans: A

MCQ 4

If a value may genuinely be missing, which design may be appropriate?

A.

`email: str | None = None`

B.

`email = AIMessage`

C.

`email = ChatOpenAI`

D.

email = PromptTemplate

Ans: A

MCQ 5

Does structured output automatically guarantee that every fact generated by the model is correct?

A. Yes

B. No

Answer

B — No

Structured output mainly gives us predictable structure and validation.

MCQ 6

Which is a good extraction instruction?

- A. Invent anything missing
- B. Use only provided information and represent unavailable optional values as null
- C. Ignore schema
- D. Always generate paragraphs

Ans: B